

Employee Stored Procedure Script

Use the following script to recreate the **Employee** and **Department** tables, define stored procedures, and insert sample data in the same sequence shown above.

Table Creation Procedures

Use these procedures to create the **Department** and **Employee** tables. The department table is created first so the employee table can reference it with a foreign key.

```
CREATE PROCEDURE SP_CreateDepartmentTable
AS
BEGIN
    CREATE TABLE Department(
        DepartmentID INT IDENTITY(1,1) PRIMARY KEY,
        DepartmentName VARCHAR(50) NOT NULL
    );
END;
```

```
CREATE PROCEDURE SP_CreateEmployeeTable
AS
BEGIN
    CREATE TABLE Employee(
        EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50),
        DepartmentID INT NOT NULL,
        CONSTRAINT FK_Employee_Department
        FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
    );
END;
```

Data Insertion Procedures

After the tables are created, use the following procedures to add department and employee records. This keeps inserts reusable and helps keep the script organized.

```
CREATE PROCEDURE SP_AddEmployee
(
```

```

    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50),
    @DepartmentID INT
)
AS
BEGIN
    INSERT INTO Employee(
        FirstName,
        LastName,
        Email,
        DepartmentID
    ) VALUES (
        @FirstName,
        @LastName,
        @Email,
        @DepartmentID
    );
END;

CREATE PROCEDURE SP_AddDepartment
(
    @DepartmentName VARCHAR(50)
)
AS
BEGIN
    INSERT INTO Department(DepartmentName)
    VALUES (@DepartmentName);
END;

```

Example Execution

Run the following statements in order to create the tables and insert a sample department and employee record.

```

EXEC SP_CreateDepartmentTable;
EXEC SP_CreateEmployeeTable;

EXEC SP_AddDepartment

```

```
@DepartmentName='HR';
```

```
EXEC SP_AddEmployee
```

```
@FirstName='John',
```

```
@LastName='Ravi',
```

```
@Email='Ravi@gmail.com',
```

```
@DepartmentID=1;
```

Class Stored Procedure Script

Use the following script to recreate the **Class** and **Student** tables, define stored procedures, and insert sample data in a clear step-by-step sequence.

Table Creation Procedures

Use these procedures to create the **Class** and **Student** tables. The class table is created first so the student table can reference it through a foreign key.

```
USE Class
```

```
DROP TABLE Class;
```

```
DROP TABLE Student;
```

```
DROP PROCEDURE CreateClassTable;
```

```
DROP PROCEDURE CreateStudentTable;
```

```
CREATE PROCEDURE SP_CreateClassTable
```

```
AS
```

```
BEGIN
```

```
    CREATE TABLE Class
```

```
    (
```

```
        ClassID INT IDENTITY(1,1) PRIMARY KEY,
```

```
        ClassName VARCHAR(50) NOT NULL
```

```
    );
```

```
END;
```

```
EXEC SP_CreateClassTable;
```

```
CREATE PROCEDURE SP_CreateStudentTable
```

```
AS
```

```

BEGIN
    CREATE TABLE Student
    (
        StudentID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50) NOT NULL,

        ClassID INT NOT NULL,

        CONSTRAINT FK_Student_Class
        FOREIGN KEY (ClassID)
        REFERENCES Class(ClassID)
    );
END;

EXEC SP_CreateStudentTable;

```

Data Insertion Procedures

After the tables are created, use the following procedures to add class and student records. This keeps the script reusable and easier to manage.

```

CREATE PROCEDURE SP_AddClass
(
    @ClassName VARCHAR(50)
)
AS
BEGIN
    INSERT INTO Class
    (
        ClassName
    )
    VALUES
    (
        @ClassName
    );
END;

```

```
CREATE PROCEDURE SP_AddStudent
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Email VARCHAR(50),
    @ClassID INT
)
AS
BEGIN
    INSERT INTO Student
    (
        FirstName,
        LastName,
        Email,
        ClassID
    ) VALUES
    (
        @FirstName,
        @LastName,
        @Email,
        @ClassID
    );
END;
```

Example Execution

Run the following statements in order to create the tables and insert a sample class and student record.

```
EXEC SP_AddClass
    @ClassName='Tagore';

EXEC SP_AddStudent
    @FirstName='Ravi',
    @LastName='Rohi',
    @Email='Ravi@gamil.com',
    @ClassID=1;
```

Outcome for Today: Employee Stored Procedure Script

Use the following script to create the **Department** and **Employee** tables, add sample records, and define procedures for retrieving and deleting data.

Table Creation Procedures

Create the department table first, then create the employee table so the foreign key relationship can be applied correctly.

USE Employee

```
CREATE PROCEDURE SP_CreateDepartmentTable
AS
BEGIN
    CREATE TABLE Department
    (
        DepartmentID INT IDENTITY(1,1) PRIMARY KEY,
        DepartmentName VARCHAR(50) NOT NULL
    );
END;
```

```
EXEC SP_CreateDepartmentTable;
```

```
CREATE PROCEDURE SP_CreateEmployeeTable
AS
BEGIN
    CREATE TABLE Employee
    (
        EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50),

        DepartmentID INT NOT NULL,

        CONSTRAINT FK_Employee_Department
        FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
    );
```

```
END;
```

```
EXEC SP_CreateEmployeeTable;
```

Data Insertion Procedures

Use these procedures to add a department record and then insert an employee linked to that department.

```
CREATE PROCEDURE SP_AddDepartment
```

```
(
```

```
    @DepartmentName VARCHAR(50)
```

```
)
```

```
AS
```

```
BEGIN
```

```
    INSERT INTO Department
```

```
(
```

```
    DepartmentName
```

```
)
```

```
VALUES
```

```
(
```

```
    @DepartmentName
```

```
);
```

```
END;
```

```
EXEC SP_AddDepartment
```

```
    @DepartmentName='IT MANAGEMENT';
```

```
CREATE PROCEDURE SP_AddEmployee
```

```
(
```

```
    @FirstName VARCHAR(50),
```

```
    @LastName VARCHAR(50),
```

```
    @Email VARCHAR(50),
```

```
    @DepartmentID INT
```

```
)
```

```
AS
```

```
BEGIN
```

```
    INSERT INTO Employee
```

```
(
```

```

    FirstName,
    LastName,
    Email,
    DepartmentID
) VALUES
(
    @FirstName,
    @LastName,
    @Email,
    @DepartmentID
);
END;

EXEC SP_AddEmployee
    @FirstName='Akhila',
    @LastName='Reddy',
    @Email='Akhila@gmail.com',
    @DepartmentID=1;

```

Retrieval and Deletion Procedures

The following procedures help retrieve all records, fetch records by ID, and remove specific department or employee entries when needed.

```

CREATE PROCEDURE SP_GetAllDepartments
AS
BEGIN
    SELECT * FROM Department;
END;

EXEC SP_GetAllDepartments;

CREATE PROCEDURE SP_GetDepartmentByID
(
    @DepartmentID INT
)
AS
BEGIN
    SELECT * FROM Department WHERE DepartmentID=@DepartmentID;

```

```
END;
```

```
EXEC SP_GetDepartmentByID
```

```
    @DepartmentID=2;
```

```
CREATE PROCEDURE SP_DeleteDepartmentByID
```

```
(
```

```
    @DepartmentID INT
```

```
)
```

```
AS
```

```
BEGIN
```

```
    DELETE FROM Department WHERE DepartmentID=@DepartmentID;
```

```
END;
```

```
EXEC SP_DeleteDepartmentByID
```

```
    @DepartmentID=3;
```

```
CREATE PROCEDURE SP_GetAllEmployees
```

```
AS
```

```
BEGIN
```

```
    SELECT * FROM Employee;
```

```
END;
```

```
EXEC SP_GetAllEmployees;
```

```
CREATE PROCEDURE SP_GetAllEmployeeByID
```

```
(
```

```
    @EmployeeID INT
```

```
)
```

```
AS
```

```
BEGIN
```

```
    SELECT * FROM Employee WHERE EmployeeID=@EmployeeID;
```

```
END;
```

```
EXEC SP_GetAllEmployeeByID
```

```
    @EmployeeID=1;
```

```
CREATE PROCEDURE SP_DeleteEmployeeByID
```

```
(  
    @EmployeeID INT  
)  
AS  
BEGIN  
    DELETE FROM Employee WHERE EmployeeID=@EmployeeID;  
END;  
  
EXEC SP_DeleteEmployeeByID  
    @EmployeeID=3;
```